

PKCERT AI & Software Development Internship

Task 17 – Neural Network Fundamentals: Perceptron, Activation Functions & Backpropagation

Objective:

Build a solid theoretical and practical foundation in the core building blocks of neural networks. Interns will implement a perceptron from scratch, analyze and compare activation functions, derive and implement forward and backward propagation manually (without relying on high-level deep learning APIs for the core computation), and apply these concepts to train a small multi-layer network on a real dataset.

Total Marks: 100

Part A – Perceptron Fundamentals (20 Marks)

1. Implement a single-layer perceptron from scratch using only NumPy (no scikit-learn, TensorFlow, or PyTorch for this part).
2. Train the perceptron on a linearly separable dataset (e.g., a subset of the Iris dataset with two classes) using the perceptron learning rule.
3. Plot the decision boundary after training and visualize how it evolves over epochs.
4. Demonstrate and explain why a single perceptron fails on a non-linearly separable dataset (e.g., XOR). Include a diagram or plot as evidence.
5. Explain, in your own words, the mathematical relationship between the perceptron and logistic regression.

Part B – Activation Functions (20 Marks)

6. Implement Sigmoid, Tanh, ReLU, Leaky ReLU, and Softmax functions from scratch (with their derivatives) using only NumPy.
7. Plot each activation function and its derivative on the same axes, over a range such as $[-10, 10]$.
8. Explain the vanishing gradient problem, identify which activation functions are most susceptible to it, and justify your answer using the derivative plots produced above.
9. Compare ReLU and Leaky ReLU: explain the 'dying ReLU' problem and how Leaky ReLU mitigates it.
10. Justify, with reasoning, which activation function you would choose for a hidden layer versus an output layer in a multi-class classification problem.

Part C – Forward & Backpropagation Mini-Project (45 Marks)

11. Select a new public classification dataset (different from datasets used in previous tasks), suitable for a small multi-layer perceptron (e.g., 2–4 input features, 2–4 output classes).
12. Manually derive the forward propagation equations for a network with at least one hidden layer, and include the derivation (matrix form) in your submission.
13. Manually derive the backpropagation gradients (using the chain rule) for the weights and biases of your chosen architecture, and include the derivation in your submission.
14. Implement forward propagation and backpropagation from scratch using only NumPy — no autograd frameworks (TensorFlow/PyTorch/Keras) permitted for this step.
15. Train your network using gradient descent (or mini-batch gradient descent), tracking and plotting the loss curve over epochs.

- Evaluate the trained model using appropriate metrics (accuracy, precision, recall, F1-score, and a confusion matrix) and compare performance against a scikit-learn MLPClassifier baseline trained on the same data.
- Perform at least one experiment varying a hyperparameter (learning rate, number of hidden units, or activation function) and report the effect on training convergence and final performance.

Part D – Analysis & Documentation (15 Marks)

- Save your final trained model's parameters (weights and biases) using pickle or joblib.
- Summarize the full pipeline: architecture chosen, activation functions used, and why.
- Summarize key findings from your hyperparameter experiment in Part C.
- Discuss at least two challenges faced while deriving or implementing backpropagation, and how you resolved them.
- Reflect on the limitations of a manually implemented MLP compared to a production-grade deep learning framework.

Assessment Breakdown

Assessment Area	Marks
Perceptron Fundamentals	20
Activation Functions	20
Forward & Backpropagation (Mini-Project)	45
Analysis & Documentation	15
Total	100

Note: For Parts A and C's core computations, use of high-level automatic-differentiation frameworks (TensorFlow, PyTorch, Keras) is not permitted — the intent is to demonstrate a working understanding of the underlying mathematics. Frameworks may be used only for the comparative baseline in Part C.